📱 Photo to PDF Converter - Complete Source Code

1. Backend - server.py

```
from fastapi import FastAPI, APIRouter, HTTPException, File, UploadFile
from fastapi.responses import FileResponse, Response
from dotenv import load_dotenv
from starlette.middleware.cors import CORSMiddleware
from motor.motor_asyncio import AsyncIOMotorClient
import os
import logging
from pathlib import Path
from pydantic import BaseModel, Field
from typing import List, Optional
import uuid
from datetime import datetime
import base64
import io
from PIL import Image
import img2pdf
import qrcode

ROOT_DIR = Path(__file__).parent
load_dotenv(ROOT_DIR / '.env')

# MongoDB connection
mongo_url = os.environ['MONGO_URL']
client = AsyncIOMotorClient(mongo_url)
db = client[os.environ['DB_NAME']]

# Create directories for storing files
UPLOAD_DIR = ROOT_DIR / "uploads"
PDF_DIR = ROOT_DIR / "pdfs"
QR_DIR = ROOT_DIR / "qrcodes"

for directory in [UPLOAD_DIR, PDF_DIR, QR_DIR]:
    directory.mkdir(exist_ok=True)

# Create the main app without a prefix
app = FastAPI()

# Create a router with the /api prefix
```

```
api_router = APIRouter(prefix="/api")

# Paper size definitions in points (1 inch = 72 points)
PAPER_SIZES = {
    "A4": (595, 842),  # 210mm x 297mm
    "A5": (420, 595),  # 148mm x 210mm
    "Letter": (612, 792),  # 8.5" x 11"
}

# Define Models
class ImageData(BaseModel):
    image_base64: str
    order: int = 0

class DocumentCreate(BaseModel):
    images: List[ImageData]
    filename: str
    paper_size: str = "A4"
    margin: int = 20  # in points
    compress: bool = True

class Document(BaseModel):
    id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    filename: str
    pdf_base64: str
    qr_base64: str
    pdf_url: str
    image_count: int
    paper_size: str
    created_at: datetime = Field(default_factory=datetime.utcnow)

class DocumentResponse(BaseModel):
    id: str
    filename: str
    pdf_base64: str
    qr_base64: str
    pdf_url: str
    image_count: int
    paper_size: str
    created_at: datetime

class DocumentUpdate(BaseModel):
    filename: str
```

```
# Helper functions
def compress_image(image: Image.Image, quality: int = 85) -> Image.Image:
    """Compress image to reduce size"""
    # Convert to RGB if needed
    if image.mode in ('RGBA', 'LA', 'P'):
        background = Image.new('RGB', image.size, (255, 255, 255))
        if image.mode == 'P':
            image = image.convert('RGBA')
        background.paste(image, mask=image.split()[-1] if image.mode in ('RGBA', 'LA') else
None)
        image = background
    elif image.mode != 'RGB':
        image = image.convert('RGB')

    # Resize if too large
    max_dimension = 2000
    if max(image.size) > max_dimension:
        ratio = max_dimension / max(image.size)
        new_size = tuple(int(dim * ratio) for dim in image.size)
        image = image.resize(new_size, Image.Resampling.LANCZOS)

    return image

def images_to_pdf_base64(
    images_base64: List[str],
    paper_size: str = "A4",
    margin: int = 20,
    compress: bool = True
) -> tuple[str, bytes]:
    """Convert multiple base64 images to PDF and return base64 PDF and bytes"""
    try:
        processed_images = []

        for img_b64 in images_base64:
            # Decode base64 image
            image_data = base64.b64decode(img_b64.split(',')[-1] if ',' in img_b64 else
img_b64)
            image = Image.open(io.BytesIO(image_data))

            # Compress if requested
            if compress:
                image = compress_image(image)
            else:
                # Still need to convert to RGB
```

```
            if image.mode in ('RGBA', 'LA', 'P'):
                background = Image.new('RGB', image.size, (255, 255, 255))
                if image.mode == 'P':
                    image = image.convert('RGBA')
                background.paste(image, mask=image.split()[-1] if image.mode in ('RGBA',
'LA') else None)
                image = background
            elif image.mode != 'RGB':
                image = image.convert('RGB')

        # Save to bytes
        img_byte_arr = io.BytesIO()
        image.save(img_byte_arr, format='JPEG', quality=95 if not compress else 85)
        img_byte_arr.seek(0)
        processed_images.append(img_byte_arr.getvalue())

    # Get paper size
    page_size = PAPER_SIZES.get(paper_size, PAPER_SIZES["A4"])

    # Convert to PDF with specified settings
    layout_fun = img2pdf.get_layout_fun(
        pagesize=page_size,
        border=(margin, margin, margin, margin)
    )

    pdf_bytes = img2pdf.convert(processed_images, layout_fun=layout_fun)
    pdf_base64 = base64.b64encode(pdf_bytes).decode('utf-8')

    return pdf_base64, pdf_bytes
  except Exception as e:
    logging.error(f"Error converting images to PDF: {str(e)}")
    raise HTTPException(status_code=400, detail=f"Error converting images to PDF:
{str(e)}")

def generate_qr_code(data: str) -> str:
  """Generate QR code and return as base64"""
  try:
    qr = qrcode.QRCode(
        version=1,
        error_correction=qrcode.constants.ERROR_CORRECT_L,
        box_size=10,
        border=4,
    )
    qr.add_data(data)
```

```
        qr.make(fit=True)

        img = qr.make_image(fill_color="black", back_color="white")

        # Convert to base64
        buffered = io.BytesIO()
        img.save(buffered, format="PNG")
        img_base64 = base64.b64encode(buffered.getvalue()).decode('utf-8')

        return img_base64
    except Exception as e:
        logging.error(f"Error generating QR code: {str(e)}")
        raise HTTPException(status_code=400, detail=f"Error generating QR code: {str(e)}")

# API Routes
@api_router.get("/")
async def root():
    return {"message": "Photo to PDF Converter API"}

@api_router.post("/upload-photo", response_model=DocumentResponse)
async def upload_photo(doc: DocumentCreate):
    """Upload multiple photos, convert to PDF, and generate QR code"""
    try:
        # Generate unique ID
        doc_id = str(uuid.uuid4())

        # Sort images by order
        sorted_images = sorted(doc.images, key=lambda x: x.order)
        images_base64 = [img.image_base64 for img in sorted_images]

        # Convert images to PDF
        pdf_base64, pdf_bytes = images_to_pdf_base64(
            images_base64,
            paper_size=doc.paper_size,
            margin=doc.margin,
            compress=doc.compress
        )

        # Save PDF to file system
        pdf_path = PDF_DIR / f"{doc_id}.pdf"
        with open(pdf_path, 'wb') as f:
            f.write(pdf_bytes)

        # Get backend URL from environment or construct it
```

```
        backend_url = os.environ.get('EXPO_PUBLIC_BACKEND_URL',
'http://localhost:8001')
        pdf_url = f"{backend_url}/api/pdf/{doc_id}"

        # Generate QR code with PDF URL
        qr_base64 = generate_qr_code(pdf_url)

        # Save QR code
        qr_path = QR_DIR / f"{doc_id}.png"
        qr_image_data = base64.b64decode(qr_base64)
        with open(qr_path, 'wb') as f:
            f.write(qr_image_data)

        # Create document object
        document = Document(
            id=doc_id,
            filename=doc.filename,
            pdf_base64=f"data:application/pdf;base64,{pdf_base64}",
            qr_base64=f"{qr_base64}",
            pdf_url=pdf_url,
            image_count=len(images_base64),
            paper_size=doc.paper_size,
            created_at=datetime.utcnow()
        )

        # Save to MongoDB
        await db.documents.insert_one(document.dict())

        return DocumentResponse(**document.dict())
    except Exception as e:
        logging.error(f"Error in upload_photo: {str(e)}")
        raise HTTPException(status_code=500, detail=f"Error processing photos: {str(e)}")

@api_router.get("/documents", response_model=List[DocumentResponse])
async def get_documents():
    """Get all documents"""
    try:
        documents = await db.documents.find().sort("created_at", -1).to_list(1000)
        return [DocumentResponse(**doc) for doc in documents]
    except Exception as e:
        logging.error(f"Error getting documents: {str(e)}")
        raise HTTPException(status_code=500, detail=f"Error retrieving documents:
{str(e)}")
```

```
@api_router.get("/pdf/{doc_id}")
async def get_pdf(doc_id: str):
    """Download PDF file"""
    try:
        pdf_path = PDF_DIR / f"{doc_id}.pdf"
        if not pdf_path.exists():
            raise HTTPException(status_code=404, detail="PDF not found")

        # Get filename from database
        doc = await db.documents.find_one({"id": doc_id})
        filename = doc.get("filename", f"document_{doc_id}") if doc else
f"document_{doc_id}"

        return FileResponse(
            path=pdf_path,
            media_type="application/pdf",
            filename=f"{filename}.pdf"
        )
    except HTTPException:
        raise
    except Exception as e:
        logging.error(f"Error getting PDF: {str(e)}")
        raise HTTPException(status_code=500, detail=f"Error retrieving PDF: {str(e)}")

@api_router.get("/qr/{doc_id}")
async def get_qr(doc_id: str):
    """Get QR code image"""
    try:
        qr_path = QR_DIR / f"{doc_id}.png"
        if not qr_path.exists():
            raise HTTPException(status_code=404, detail="QR code not found")

        return FileResponse(
            path=qr_path,
            media_type="image/png",
            filename=f"qr_{doc_id}.png"
        )
    except HTTPException:
        raise
    except Exception as e:
        logging.error(f"Error getting QR code: {str(e)}")
        raise HTTPException(status_code=500, detail=f"Error retrieving QR code: {str(e)}")

@api_router.patch("/document/{doc_id}", response_model=DocumentResponse)
```

```
async def update_document(doc_id: str, update: DocumentUpdate):
    """Update document (rename)"""
    try:
        result = await db.documents.find_one_and_update(
            {"id": doc_id},
            {"$set": {"filename": update.filename}},
            return_document=True
        )

        if not result:
            raise HTTPException(status_code=404, detail="Document not found")

        return DocumentResponse(**result)
    except HTTPException:
        raise
    except Exception as e:
        logging.error(f"Error updating document: {str(e)}")
        raise HTTPException(status_code=500, detail=f"Error updating document: {str(e)}")

@api_router.delete("/document/{doc_id}")
async def delete_document(doc_id: str):
    """Delete a document"""
    try:
        # Delete from database
        result = await db.documents.delete_one({"id": doc_id})

        if result.deleted_count == 0:
            raise HTTPException(status_code=404, detail="Document not found")

        # Delete files
        pdf_path = PDF_DIR / f"{doc_id}.pdf"
        qr_path = QR_DIR / f"{doc_id}.png"

        if pdf_path.exists():
            pdf_path.unlink()
        if qr_path.exists():
            qr_path.unlink()

        return {"message": "Document deleted successfully"}
    except HTTPException:
        raise
    except Exception as e:
        logging.error(f"Error deleting document: {str(e)}")
        raise HTTPException(status_code=500, detail=f"Error deleting document: {str(e)}")
```

```
# Include the router in the main app
app.include_router(api_router)

app.add_middleware(
    CORSMiddleware,
    allow_credentials=True,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# Configure logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)
logger = logging.getLogger(__name__)

@app.on_event("shutdown")
async def shutdown_db_client():
    client.close()
```

2. Backend - requirements.txt

```
fastapi==0.110.1
uvicorn==0.25.0
motor==3.3.1
pymongo==4.5.0
Pillow==12.1.1
img2pdf==0.6.3
qrcode==8.2
python-dotenv==1.2.1
pydantic==2.12.5
python-multipart==0.0.20
```

3. Frontend - index.tsx (Main Screen)

```
import React, { useState, useEffect } from 'react';
import {
  Text,
  View,
  StyleSheet,
```

```
  TouchableOpacity,
  Image,
  ScrollView,
  ActivityIndicator,
  Alert,
  TextInput,
  Switch,
  useColorScheme,
} from 'react-native';
import * as ImagePicker from 'expo-image-picker';
import * as FileSystem from 'expo-file-system';
import * as Sharing from 'expo-sharing';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';
import { useRouter } from 'expo-router';
import AsyncStorage from '@react-native-async-storage/async-storage';

const EXPO_PUBLIC_BACKEND_URL = process.env.EXPO_PUBLIC_BACKEND_URL;

interface SelectedImage {
  uri: string;
  base64: string;
  order: number;
}

export default function Index() {
  const systemColorScheme = useColorScheme();
  const [isDarkMode, setIsDarkMode] = useState(systemColorScheme === 'dark');
  const [selectedImages, setSelectedImages] = useState<SelectedImage[]>([]);
  const [pdfBase64, setPdfBase64] = useState<string | null>(null);
  const [qrBase64, setQrBase64] = useState<string | null>(null);
  const [loading, setLoading] = useState(false);
  const [currentDocId, setCurrentDocId] = useState<string | null>(null);
  const [filename, setFilename] = useState('document');
  const [paperSize, setPaperSize] = useState<'A4' | 'A5' | 'Letter'>('A4');
  const [margin, setMargin] = useState(20);
  const [compress, setCompress] = useState(true);
  const [showSettings, setShowSettings] = useState(false);
  const router = useRouter();

  // Load dark mode preference
  useEffect(() => {
    loadDarkModePreference();
  }, []);
```

```
  const loadDarkModePreference = async () => {
    try {
      const saved = await AsyncStorage.getItem('darkMode');
      if (saved !== null) {
        setIsDarkMode(saved === 'true');
      }
    } catch (error) {
      console.error('Error loading dark mode:', error);
    }
  };

  const toggleDarkMode = async () => {
    const newValue = !isDarkMode;
    setIsDarkMode(newValue);
    try {
      await AsyncStorage.setItem('darkMode', String(newValue));
    } catch (error) {
      console.error('Error saving dark mode:', error);
    }
  };

  const theme = isDarkMode ? darkTheme : lightTheme;

  // Share to WhatsApp
  const shareToWhatsApp = async (type: 'pdf' | 'qr') => {
    if (!currentDocId) return;

    try {
      const url = type === 'pdf'
        ? `${EXPO_PUBLIC_BACKEND_URL}/api/pdf/${currentDocId}`
        : `${EXPO_PUBLIC_BACKEND_URL}/api/qr/${currentDocId}`;

      const text = type === 'pdf'
        ? `📄 PDF: ${filename}\n\n${url}`
        : `🔗 QR Code: ${filename}\n\n${url}`;

      const whatsappUrl =
`https://api.whatsapp.com/send?text=${encodeURIComponent(text)}`;

      if (typeof window !== 'undefined') {
        window.open(whatsappUrl, '_blank');
      }
    } catch (error) {
```

```
    console.error('WhatsApp share error:', error);
    Alert.alert('Error', 'WhatsApp पर share नहीं हो पाया');
  }
};

// Share via Email
const shareViaEmail = async (type: 'pdf' | 'qr') => {
  if (!currentDocId) return;

  try {
    const url = type === 'pdf'
      ? `${EXPO_PUBLIC_BACKEND_URL}/api/pdf/${currentDocId}`
      : `${EXPO_PUBLIC_BACKEND_URL}/api/qr/${currentDocId}`;

    const subject = type === 'pdf'
      ? `PDF: ${filename}`
      : `QR Code: ${filename}`;

    const body = `${type === 'pdf' ? '📄 PDF Document' : '🔗 QR Code'}\n\nFilename:
${filename}\n\nDownload Link:\n${url}`;

    const mailtoUrl =
`mailto:?subject=${encodeURIComponent(subject)}&body=${encodeURIComponent(bod
y)}`;

    if (typeof window !== 'undefined') {
      window.open(mailtoUrl, '_blank');
    }
  } catch (error) {
    console.error('Email share error:', error);
    Alert.alert('Error', 'Email भेजने में problem आई');
  }
};

// Share to Google Drive (opens Drive with link)
const shareToDrive = async (type: 'pdf' | 'qr') => {
  if (!currentDocId) return;

  try {
    const url = type === 'pdf'
      ? `${EXPO_PUBLIC_BACKEND_URL}/api/pdf/${currentDocId}`
      : `${EXPO_PUBLIC_BACKEND_URL}/api/qr/${currentDocId}`;

    Alert.alert(
```

```
      '📤 Share to Google Drive',
      'File को save करने के लिए:\n\n1. Link copy करें\n2. Drive app खोलें\n3. Upload from URL
option use करें\n\nLink:',
      [
        {
          text: '📋 Link Copy करें',
          onPress: async () => {
            try {
              if (typeof navigator !== 'undefined' && navigator.clipboard) {
                await navigator.clipboard.writeText(url);
                Alert.alert('✅ Success', 'Link copied! Drive में paste करें');
              } else {
                Alert.alert('Link', url);
              }
            } catch (error) {
              Alert.alert('Link', url);
            }
          },
        },
        {
          text: '🌐 Browser में खोलें',
          onPress: () => {
            if (typeof window !== 'undefined') {
              window.open(url, '_blank');
            }
          },
        },
        { text: 'Cancel', style: 'cancel' },
      ]
    );
  } catch (error) {
    console.error('Drive share error:', error);
    Alert.alert('Error', 'Share नहीं हो पाया');
  }
};

// Request permissions
const requestPermissions = async () => {
  const { status } = await ImagePicker.requestMediaLibraryPermissionsAsync();
  if (status !== 'granted') {
    Alert.alert(
      'Permission Required',
      'हमें gallery access की ज़रूरत है photos select करने के लिए!'
    );
```

```
    return false;
  }
  return true;
};

// Pick multiple images
const pickImages = async () => {
  const hasPermission = await requestPermissions();
  if (!hasPermission) return;

  try {
    const result = await ImagePicker.launchImageLibraryAsync({
      mediaTypes: ['images'],
      allowsMultipleSelection: true,
      quality: 1,
      base64: true,
    });

    if (!result.canceled && result.assets) {
      const newImages: SelectedImage[] = result.assets.map((asset, index) => ({
        uri: asset.uri,
        base64: asset.base64 || '',
        order: selectedImages.length + index,
      }));
      setSelectedImages([...selectedImages, ...newImages]);
    }
  } catch (error) {
    console.error('Error picking images:', error);
    Alert.alert('Error', 'Failed to pick images');
  }
};

// Take photo with camera
const takePhoto = async () => {
  const { status } = await ImagePicker.requestCameraPermissionsAsync();
  if (status !== 'granted') {
    Alert.alert(
      'Permission Required',
      'हमें camera access की ज़रूरत है photo लेने के लिए!'
    );
    return;
  }

  try {
```

```
      const result = await ImagePicker.launchCameraAsync({
        allowsEditing: false,
        quality: 1,
        base64: true,
      });

      if (!result.canceled && result.assets[0]) {
        const newImage: SelectedImage = {
          uri: result.assets[0].uri,
          base64: result.assets[0].base64 || '',
          order: selectedImages.length,
        };
        setSelectedImages([...selectedImages, newImage]);
      }
    } catch (error) {
      console.error('Error taking photo:', error);
      Alert.alert('Error', 'Failed to take photo');
    }
  };

  // Remove image
  const removeImage = (index: number) => {
    const newImages = selectedImages.filter((_, i) => i !== index);
    const reorderedImages = newImages.map((img, i) => ({ ...img, order: i }));
    setSelectedImages(reorderedImages);
  };

  // Move image up
  const moveImageUp = (index: number) => {
    if (index === 0) return;
    const newImages = [...selectedImages];
    [newImages[index - 1], newImages[index]] = [newImages[index], newImages[index -
1]];
    const reorderedImages = newImages.map((img, i) => ({ ...img, order: i }));
    setSelectedImages(reorderedImages);
  };

  // Move image down
  const moveImageDown = (index: number) => {
    if (index === selectedImages.length - 1) return;
    const newImages = [...selectedImages];
    [newImages[index], newImages[index + 1]] = [newImages[index + 1],
newImages[index]];
    const reorderedImages = newImages.map((img, i) => ({ ...img, order: i }));
```

```
  setSelectedImages(reorderedImages);
};

// Generate PDF
const generatePDF = async () => {
  if (selectedImages.length === 0) {
    Alert.alert('Error', 'कृपया कम से कम एक photo select करें!');
    return;
  }

  if (!filename.trim()) {
    Alert.alert('Error', 'कृपया filename enter करें!');
    return;
  }

  setLoading(true);
  try {
    const imageData = selectedImages.map((img) => ({
      image_base64: img.base64,
      order: img.order,
    }));

    const response = await fetch(`${EXPO_PUBLIC_BACKEND_URL}/api/upload-photo`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        images: imageData,
        filename: filename.trim(),
        paper_size: paperSize,
        margin: margin,
        compress: compress,
      }),
    });

    if (!response.ok) {
      throw new Error('Failed to generate PDF');
    }

    const data = await response.json();
    setPdfBase64(data.pdf_base64);
    setQrBase64(data.qr_base64);
    setCurrentDocId(data.id);
```

```
    Alert.alert('✅ Success', `PDF successfully बना दिया गया!\n${selectedImages.length}
photos से`);
  } catch (error) {
    console.error('Error generating PDF:', error);
    Alert.alert('❌ Error', 'PDF बनाने में problem हुई!');
  } finally {
    setLoading(false);
  }
};

// Download PDF
const downloadPDF = async () => {
  if (!currentDocId) {
    Alert.alert('Error', 'PDF ID नहीं मिला!');
    return;
  }

  try {
    const pdfUrl = `${EXPO_PUBLIC_BACKEND_URL}/api/pdf/${currentDocId}`;

    Alert.alert(
      '📄 PDF Download',
      'PDF को download/share करने के लिए option चुनें:',
      [
        {
          text: '🌐 Browser में खोलें',
          onPress: async () => {
            if (typeof window !== 'undefined') {
              window.open(pdfUrl, '_blank');
            }
          },
        },
        {
          text: '📤 Share करें',
          onPress: async () => {
            try {
              if (!pdfBase64) return;
              const base64Data = pdfBase64.split(',')[1] || pdfBase64;
              const fileUri = FileSystem.documentDirectory + `${filename}.pdf`;

              await FileSystem.writeAsStringAsync(fileUri, base64Data, {
                encoding: FileSystem.EncodingType.Base64,
              });
```

```
            if (await Sharing.isAvailableAsync()) {
              await Sharing.shareAsync(fileUri, {
                mimeType: 'application/pdf',
                dialogTitle: 'PDF Share करें',
                UTI: 'com.adobe.pdf',
              });
            }
          } catch (error) {
            console.error('Share error:', error);
            Alert.alert('Error', 'Share नहीं हो पाया. Browser से download करें.');
          }
        },
      },
      {
        text: '❌ Cancel',
        style: 'cancel',
      },
    ]
  );
} catch (error) {
  console.error('Error downloading PDF:', error);
  Alert.alert('Error', 'PDF download में problem आई!');
}
};

// Download QR code
const downloadQR = async () => {
  if (!currentDocId) {
    Alert.alert('Error', 'QR code ID नहीं मिला!');
    return;
  }

  try {
    const qrUrl = `${EXPO_PUBLIC_BACKEND_URL}/api/qr/${currentDocId}`;

    Alert.alert(
      '🔗 QR Code Download',
      'QR code को download/share करने के लिए option चुनें:',
      [
        {
          text: '🌐 Browser में खोलें',
          onPress: () => {
            if (typeof window !== 'undefined') {
              window.open(qrUrl, '_blank');
```

```
          }
        },
      },
      {
        text: '📤 Share करें',
        onPress: async () => {
          try {
            if (!qrBase64) return;
            const base64Data = qrBase64.split(',')[1] || qrBase64;
            const fileUri = FileSystem.documentDirectory + `qr_${filename}.png`;

            await FileSystem.writeAsStringAsync(fileUri, base64Data, {
              encoding: FileSystem.EncodingType.Base64,
            });

            if (await Sharing.isAvailableAsync()) {
              await Sharing.shareAsync(fileUri, {
                mimeType: 'image/png',
                dialogTitle: 'QR Code Share करें',
              });
            }
          } catch (error) {
            console.error('Share error:', error);
            Alert.alert('Error', 'Share नहीं हो पाया. Browser से download करें.');
          }
        },
      },
      {
        text: '❌ Cancel',
        style: 'cancel',
      },
    ]
  );
} catch (error) {
  console.error('Error downloading QR:', error);
  Alert.alert('Error', 'QR download में problem आई!');
}
};

// Reset
const reset = () => {
  setSelectedImages([]);
  setPdfBase64(null);
  setQrBase64(null);
```

```
    setCurrentDocId(null);
    setFilename('document');
    setShowSettings(false);
  };

  return (
    <SafeAreaView style={[styles.container, { backgroundColor: theme.background }]}>
      <ScrollView contentContainerStyle={styles.scrollContent}>
        {/* Header */}
        <View style={styles.header}>
          <View style={styles.headerTop}>
            <View style={styles.headerLeft}>
              <Ionicons name="document-text" size={40} color={theme.primary} />
              <View>
                <Text style={[styles.title, { color: theme.text }]}>Photo to PDF</Text>
                <Text style={[styles.subtitle, { color: theme.textSecondary }]}>
                  Multiple photos से PDF बनाएं
                </Text>
              </View>
            </View>
            <TouchableOpacity onPress={toggleDarkMode} style={styles.themeToggle}>
              <Ionicons
                name={isDarkMode ? 'sunny' : 'moon'}
                size={24}
                color={theme.primary}
              />
            </TouchableOpacity>
          </View>
        </View>

        {/* ... Rest of the UI components ... */}
        {/* (Full code available in the complete source) */}

      </ScrollView>
    </SafeAreaView>
  );
}

const lightTheme = {
  background: '#f8fafc',
  card: '#ffffff',
  text: '#1e293b',
  textSecondary: '#64748b',
  primary: '#6366f1',
```

```
  primaryLight: '#ede9fe',
  border: '#e2e8f0',
};

const darkTheme = {
  background: '#0f172a',
  card: '#1e293b',
  text: '#f1f5f9',
  textSecondary: '#94a3b8',
  primary: '#818cf8',
  primaryLight: '#312e81',
  border: '#334155',
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
  },
  scrollContent: {
    padding: 16,
  },
  header: {
    marginBottom: 24,
  },
  headerTop: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
  },
  headerLeft: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 12,
    flex: 1,
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
  },
  subtitle: {
    fontSize: 14,
    marginTop: 2,
  },
  themeToggle: {
```

```
    padding: 8,
  },
  // ... more styles
});
```

4. Frontend - history.tsx (History Screen)

```
import React, { useState, useEffect } from 'react';
import {
  Text,
  View,
  StyleSheet,
  FlatList,
  TouchableOpacity,
  Image,
  ActivityIndicator,
  Alert,
  RefreshControl,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';
import { useRouter } from 'expo-router';
import * as FileSystem from 'expo-file-system';
import * as Sharing from 'expo-sharing';

const EXPO_PUBLIC_BACKEND_URL = process.env.EXPO_PUBLIC_BACKEND_URL;

interface Document {
  id: string;
  filename: string;
  pdf_base64: string;
  qr_base64: string;
  pdf_url: string;
  created_at: string;
}

export default function History() {
  const [documents, setDocuments] = useState<Document[]>([]);
  const [loading, setLoading] = useState(true);
  const [refreshing, setRefreshing] = useState(false);
  const router = useRouter();

  useEffect(() => {
    fetchDocuments();
  }, []);
```

```
const fetchDocuments = async () => {
  try {
    const response = await fetch(`${EXPO_PUBLIC_BACKEND_URL}/api/documents`);
    if (!response.ok) {
      throw new Error('Failed to fetch documents');
    }
    const data = await response.json();
    setDocuments(data);
  } catch (error) {
    console.error('Error fetching documents:', error);
    Alert.alert('Error', 'Failed to load documents');
  } finally {
    setLoading(false);
    setRefreshing(false);
  }
};

const onRefresh = () => {
  setRefreshing(true);
  fetchDocuments();
};

const downloadPDF = async (doc: Document) => {
  try {
    const base64Data = doc.pdf_base64.split(',')[1] || doc.pdf_base64;
    const fileUri = FileSystem.documentDirectory + `${doc.filename}.pdf`;

    await FileSystem.writeAsStringAsync(fileUri, base64Data, {
      encoding: FileSystem.EncodingType.Base64,
    });

    if (await Sharing.isAvailableAsync()) {
      await Sharing.shareAsync(fileUri, {
        mimeType: 'application/pdf',
        dialogTitle: 'Save or Share PDF',
      });
    } else {
      Alert.alert('Success', 'PDF saved to device!');
    }
  } catch (error) {
    console.error('Error downloading PDF:', error);
    Alert.alert('Error', 'Failed to download PDF');
  }
```

```
};

const downloadQR = async (doc: Document) => {
  try {
    const base64Data = doc.qr_base64.split(',')[1] || doc.qr_base64;
    const fileUri = FileSystem.documentDirectory + `qr_${doc.id}.png`;

    await FileSystem.writeAsStringAsync(fileUri, base64Data, {
      encoding: FileSystem.EncodingType.Base64,
    });

    if (await Sharing.isAvailableAsync()) {
      await Sharing.shareAsync(fileUri, {
        mimeType: 'image/png',
        dialogTitle: 'Save or Share QR Code',
      });
    } else {
      Alert.alert('Success', 'QR code saved to device!');
    }
  } catch (error) {
    console.error('Error downloading QR:', error);
    Alert.alert('Error', 'Failed to download QR code');
  }
};

const deleteDocument = async (docId: string) => {
  Alert.alert(
    'Delete Document',
    'Are you sure you want to delete this document?',
    [
      { text: 'Cancel', style: 'cancel' },
      {
        text: 'Delete',
        style: 'destructive',
        onPress: async () => {
          try {
            const response = await fetch(
              `${EXPO_PUBLIC_BACKEND_URL}/api/document/${docId}`,
              { method: 'DELETE' }
            );
            if (!response.ok) {
              throw new Error('Failed to delete document');
            }
            Alert.alert('Success', 'Document deleted successfully');
```

```
          fetchDocuments();
        } catch (error) {
          console.error('Error deleting document:', error);
          Alert.alert('Error', 'Failed to delete document');
        }
      },
    },
  ]
);
};

const formatDate = (dateString: string) => {
  const date = new Date(dateString);
  return date.toLocaleDateString() + ' ' + date.toLocaleTimeString();
};

const renderDocument = ({ item }: { item: Document }) => (
  <View style={styles.documentCard}>
    <View style={styles.cardHeader}>
      <View style={styles.headerLeft}>
        <Ionicons name="document-text" size={24} color="#6366f1" />
        <View style={styles.headerInfo}>
          <Text style={styles.filename} numberOfLines={1}>
            {item.filename}
          </Text>
          <Text style={styles.date}>{formatDate(item.created_at)}</Text>
        </View>
      </View>
      <TouchableOpacity onPress={() => deleteDocument(item.id)}>
        <Ionicons name="trash" size={20} color="#ef4444" />
      </TouchableOpacity>
    </View>

    <View style={styles.previewContainer}>
      <Image source={{ uri: item.qr_base64 }} style={styles.qrPreview} />
    </View>

    <View style={styles.actionButtons}>
      <TouchableOpacity
        style={styles.actionButton}
        onPress={() => downloadPDF(item)}
      >
        <Ionicons name="download" size={18} color="white" />
        <Text style={styles.actionButtonText}>PDF</Text>
```

```
      </TouchableOpacity>

      <TouchableOpacity
        style={styles.actionButton}
        onPress={() => downloadQR(item)}
      >
        <Ionicons name="qr-code" size={18} color="white" />
        <Text style={styles.actionButtonText}>QR</Text>
      </TouchableOpacity>
    </View>
  </View>
);

if (loading) {
  return (
    <SafeAreaView style={styles.container}>
      <View style={styles.loadingContainer}>
        <ActivityIndicator size="large" color="#6366f1" />
        <Text style={styles.loadingText}>Loading documents...</Text>
      </View>
    </SafeAreaView>
  );
}

return (
  <SafeAreaView style={styles.container}>
    <View style={styles.header}>
      <TouchableOpacity onPress={() => router.back()} style={styles.backButton}>
        <Ionicons name="arrow-back" size={24} color="#1e293b" />
      </TouchableOpacity>
      <Text style={styles.title}>History</Text>
      <View style={{ width: 40 }} />
    </View>

    {documents.length === 0 ? (
      <View style={styles.emptyContainer}>
        <Ionicons name="document-text-outline" size={64} color="#cbd5e1" />
        <Text style={styles.emptyText}>No documents yet</Text>
        <Text style={styles.emptySubtext}>Start by converting your first photo to
PDF</Text>
      </View>
    ) : (
      <FlatList
        data={documents}
```

```
        renderItem={renderDocument}
        keyExtractor={(item) => item.id}
        contentContainerStyle={styles.listContent}
        refreshControl={
          <RefreshControl refreshing={refreshing} onRefresh={onRefresh} />
        }
      />
    )}
  </SafeAreaView>
 );
}

const styles = StyleSheet.create({
 container: {
  flex: 1,
  backgroundColor: '#f8fafc',
 },
 header: {
  flexDirection: 'row',
  alignItems: 'center',
  justifyContent: 'space-between',
  paddingHorizontal: 20,
  paddingVertical: 16,
  backgroundColor: 'white',
  borderBottomWidth: 1,
  borderBottomColor: '#e2e8f0',
 },
 backButton: {
  width: 40,
  height: 40,
  alignItems: 'center',
  justifyContent: 'center',
 },
 title: {
  fontSize: 20,
  fontWeight: 'bold',
  color: '#1e293b',
 },
 // ... more styles
});
```

5. Frontend - _layout.tsx (App Layout)

```
import { Stack } from 'expo-router';
```

```
import { useEffect } from 'react';
import { Platform } from 'react-native';

export default function RootLayout() {
  useEffect(() => {
    // Register service worker for PWA
    if (Platform.OS === 'web' && 'serviceWorker' in navigator) {
      navigator.serviceWorker.register('/sw.js').catch((error) => {
        console.log('Service Worker registration failed:', error);
      });
    }
  }, []);

  return (
    <Stack
      screenOptions={{
        headerShown: false,
        animation: 'slide_from_right',
      }}
    >
      <Stack.Screen name="index" />
      <Stack.Screen name="history" />
    </Stack>
  );
}
```

6. Configuration Files

- ### Backend .env

```
MONGO_URL=mongodb://localhost:27017
DB_NAME=photo_to_pdf_db
EXPO_PUBLIC_BACKEND_URL=https://your-domain.com
```

- ### Frontend .env

```
EXPO_PUBLIC_BACKEND_URL=https://your-domain.com
```

- ### package.json (Frontend)

```
{
  "name": "photo-to-pdf-frontend",
  "version": "1.0.0",
  "dependencies": {
    "expo": "54.0.33",
    "expo-router": "~6.0.22",
```

```
    "expo-image-picker": "55.0.12",
    "expo-file-system": "55.0.10",
    "expo-sharing": "55.0.11",
    "@react-native-async-storage/async-storage": "3.0.1",
    "react-native": "0.81.5",
    "react-native-svg": "15.15.3",
    "@expo/vector-icons": "^14.1.0",
    "react-native-safe-area-context": "^5.4.0"
  }
}
```

7. Installation Guide

- ## Backend Setup

```
cd backend
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
uvicorn server: app --host 0.0.0.0 --port 8001 --reload
```

- ## Frontend Setup

```
cd frontend
yarn install
yarn start
# or for web
yarn web
```

- ## MongoDB

```
mongod --dbpath /data/db
```